Object Oriented Programming (CSEG1044)
Unit 02 – Lecture 03 Notes
Access Modifiers + Packages + Interface Intro

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **2**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**

7.1 Access control (what each means) . . . 3

7.2 Packages: why and how . . . 4

7.3 Interface intro: contract view . . . 4

8 Running Project Connection **4**

9 Common Mistakes and Confusions **5**

10 Worked Example: packages + interface (small) **5**

11 Demo / Observation Task **6**

12 Practice Check (with brief solutions) **6**

13 Self-Study Practice (no solutions) **6**

14 Further Study **7**

15 Exit Questions (with sample answers) **7**

1 Lecture Snapshot

- **Unit focus:** Classes, Inheritance, Packages, and Interfaces
- **Main new ideas:** Access control (what each means), Packages: why and how, Interface intro: contract view
- **Course thread:** Use Access Modifiers + Packages + Interface Intro to strengthen the domain model, APIs, and access rules inside Campus Resource Manager.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

2 Lecture Overview

As your codebase grows, you need two things:

- **control visibility** of classes/fields/methods (to prevent misuse),
- **organize code** into packages (so files and names stay manageable).

We also introduce the **interface** idea as a contract to reduce coupling.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Encapsulation**: Access modifiers are the concrete mechanism for the encapsulation idea. Main reference: `Unit 02 / Lecture 01 slides/notes`.
- **Class organization**: Packages are easier to understand once class members and static utilities are familiar. Main reference: `Unit 02 / Lecture 02 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Choose correct access modifiers to protect a class design.
- Organize code using packages and imports.
- Explain interfaces as contracts and write a basic interface example.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Access control (what each means)* → *Packages: why and how* → *Interface intro: contract view.*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Package**: namespace that groups related classes (also maps to folder structure).
- **Package-private (default)**: visible only inside the same package.
- **Protected**: visible in same package and in subclasses (even in other packages).
- **Public API**: the classes/methods you intentionally expose for others to use.
- **Coupling**: how strongly one part of code depends on another.

7 Detailed Notes

7.1 Access control (what each means)

Visibility rules (simplified for day-to-day use):

- `private`: only inside the same class.
- **default** (no keyword): inside the same package.
- `protected`: inside the same package, and also in subclasses.
- `public`: everywhere.

Rule of thumb.

- Keep fields `private`.
- Expose methods only if necessary.
- Use package-private for helper classes that should not be part of the public API.

7.2 Packages: why and how

Why packages?

- Avoid name conflicts (`Student` in two different domains).
- Improve organization and readability.
- Control visibility using package-private access.

How to declare a package: the first non-comment line should be:

- `package com.upes.crm.model;`

The folder path should match:

- `com/upes/crm/model/Student.java`

Imports:

- Use `import` to avoid writing fully-qualified names.
- Example: `import java.util.List;`

7.3 Interface intro: contract view

An interface defines **what operations** a class must provide, without specifying how. This allows you to program to an interface (a contract), not to a concrete class.

Key benefit: reduced coupling. Your code depends on an abstract contract, so implementations can change without changing the caller.

We will cover interfaces deeply later (multiple files, design decisions, and real use cases).

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

8 Running Project Connection

Use Access Modifiers + Packages + Interface Intro to strengthen the domain model, APIs, and access rules inside Campus Resource Manager.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.

- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Using public as a shortcut whenever code fails to compile.
- Creating packages with no responsibility boundary.
- Treating an interface as just a weaker class.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

10 Worked Example: packages + interface (small)

File 1: `com/upes/demo/pay/Payable.java`

```java
package com.upes.demo.pay;

public interface Payable {
    double amountDue();
}
```

File 2: `com/upes/demo/pay/Invoice.java`

```java
package com.upes.demo.pay;

public class Invoice implements Payable {
    private double total;

    public Invoice(double total) {
        if (total < 0) throw new IllegalArgumentException("total");
        this.total = total;
    }

    @Override
    public double amountDue() {
        return total;
    }
}
```

File 3: `com/upes/demo/app/Main.java`

```java
package com.upes.demo.app;

import com.upes.demo.pay.Invoice;
import com.upes.demo.pay.Payable;

public class Main {
    public static void main(String[] args) {
        Payable p = new Invoice(2500.0);
        System.out.println(p.amountDue());
    }
}
```

The line `Payable p = new Invoice(...)` is used here only to show the **contract view**: the caller depends on `amountDue()`, not on invoice internals. The runtime dispatch explanation comes in Lecture 04.

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Access control (what each means)* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. Which access modifier should be used for most fields? **Answer:** `private`.
2. If a member has no access modifier, where is it visible? **Answer:** within the same package.
3. Why do we use packages? **Answer:** organization + avoid name conflicts + control visibility.

13 Self-Study Practice (no solutions)

1. Create package `com.upes.bank` with classes `Account` and `Customer`.
2. Write an interface `Printable` with method `print()` and implement it in two classes.
3. Make a helper class package-private and observe where it can/can't be used.

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.
- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed April 1, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. What is the difference between `private` and package-private?

Sample answer: `private` is visible only inside the same class. Package-private (default) is visible to any class in the same package.

Q2. Why are interfaces useful?

Sample answer: Interfaces define a contract and reduce coupling. Code written against an interface can work with multiple implementations without changing the caller.

Q3. Why do packages matter even in a small project?

Sample answer: They control visibility, communicate architecture, and reduce naming collisions.